MAPA DO PROJETO · OUTUBRO DE 2026

IAEmPS2

Um mestre invisível por inteligência artificial para jogos de PlayStation 2 em emulador. Ele observa a partida pela memória do emulador e intervém, dentro de limites declarados por jogo, para a experiência ficar mais viva: alivia no perigo real, aperta quando o jogador domina, e muitas vezes não faz nada. Referência de visão, escopo, decisões e roteiro; Casos de Uso e Blueprint Técnico derivam dele.

Nome provisório. "IAEmPS2" é o nome da pasta do protótipo e segue como nome de trabalho; trocar é decisão do cliente e não altera nada além do título. **Origem.** Protótipo sobre Final Fantasy XII, de fevereiro a março de 2026; o que provou e ensinou está na seção Origem.

10	3	10	12
módulos funcionais	fases de entrega	regras de negócio	decisões, 3 fechadas e 9 propostas

MÓDULO A

Emulador e memória

fase 1

A ponte com o emulador: localizar a memória do jogo pela interface oficial, identificar o jogo em execução, ler e escrever com segurança.

MÓDULO B

Perfil de jogo

fase 1

O mapa de significado de cada jogo: variáveis com endereço, tipo, faixa e rótulo, ações permitidas e o que nunca tocar. Criado, importado, validado e versionado.

MÓDULO C

Mestre

fase 1

Observar o estado, decidir uma ação e um motivo, respeitar o ritmo. Tom e intensidade configuráveis antes da sessão.

MÓDULO D

Ações e limites

fase 1

O catálogo de ações derivado do perfil, a validação de faixa, o intervalo mínimo, a reversão da última ação e a pausa.

MÓDULO E

Sessão e overlay

fase 1

Abrir a sessão com um perfil, mostrar estado, ação e motivo na tela, pausar, retomar, encerrar com resumo.

MÓDULO F

Registro

fase 1

Toda observação e intervenção gravadas com estado antes, ação, estado depois e motivo. Reproduzível e exportável.

MÓDULO G

Processos automáticos

fase 1

Validar o perfil contra o jogo em execução, medir custo e latência, tratar falha do serviço de linguagem e do emulador.

MÓDULO H

Catálogo de perfis

fase 2

Perfis compartilháveis entre jogadores, descoberta assistida de variáveis e importação de fontes públicas de mapas.

MÓDULO I

Entidades e mundo

fase 2

Posição de inimigos e personagens: aproximar, afastar, reunir. Exige mapa de entidades por jogo.

MÓDULO J

Visão e controle

fase 3

O mestre vê a tela e aciona o controle. Outro tipo de intervenção, só depois que as anteriores estiverem provadas.

VISÃO

Um mestre que decide, um perfil que limita, uma plataforma que executa

O produto é um diretor de jogo por inteligência artificial: ele não joga e não obedece a pedidos do jogador. Ele lê o estado da partida e molda o que acontece, como um mestre de mesa que percebe o grupo entediado e solta uma emboscada, ou percebe o desespero e deixa um frasco de cura na gaveta. O que ele não faz é tocar memória diretamente: todo endereço, faixa e escrita vivem no perfil do jogo e na plataforma, que são código determinístico.

Mestre (serviço de linguagem)

- Recebe o estado resumido pelo perfil
- Decide uma ação do catálogo e um motivo
- Pode decidir "nada"
- Não conhece nenhum endereço
- Trocável por configuração

Perfil de jogo

- Variável = endereço, tipo, faixa, rótulo
- Ações permitidas e seus limites
- Lista do que nunca se toca
- Imutável durante a sessão
- Versionado e validado contra o jogo

Plataforma

- Lê e escreve a memória pela ponte
- Valida cada escrita contra o perfil
- Relê o estado antes de escrever
- Registra tudo com motivo
- Testável sem serviço de linguagem

A PRIMEIRA VERSÃO É

- Um jogador, um jogo, um perfil pronto: Final Fantasy XII, o banco de ensaio.
- Overlay sobre a janela do emulador; produto em português do Brasil.
- O ciclo completo: abrir a sessão, o mestre observa, decide e age dentro do perfil, o jogador vê, pausa ou reverte, encerra com resumo.
- Só valores numéricos: vida, energia, dinheiro, atributos, itens contáveis.
- Toda intervenção registrada e reproduzível.

A PRIMEIRA VERSÃO NÃO É

- Catálogo de perfis compartilhado entre jogadores (fase 2).
- Mover inimigos ou personagens no mapa (fase 2).
- Ver a tela ou apertar botões (fase 3).
- Trainer com conversa: o jogador não pede efeito.
- Jogador por inteligência artificial, nem outro emulador ou console.

Por que começar por valores numéricos. Uma variável numérica tem gabarito: valor antes, valor depois, faixa válida. Dá para testar cada intervenção sem opinião, e é exatamente o que as fontes públicas de mapas de memória já documentam para centenas de jogos. Entidades no mapa não têm esse gabarito: a tentativa do protótipo de achá-las por varredura devolveu trinta candidatas, das quais duas pareciam reais. Elas exigem engenharia reversa por jogo e entram quando o produto já estiver provado no que é mensurável.

ATORES

Quem usa e o que fica de fora

Ator é quem inicia ou recebe alguma coisa da plataforma. Na primeira versão, jogador e mapeador são a mesma pessoa, que também é o cliente deste projeto: ele aprova cada documento e cada sessão jogada é um teste de aceitação. A separação existe desde já porque o produto é para outros jogadores também, e a maioria deles só vai jogar.

Jogador

`pessoa · ator principal nos módulos C, E`

Escolhe o perfil, configura tom e intensidade, joga, vê o que o mestre fez, pausa, reverte e encerra.

Mapeador

`pessoa · ator principal nos módulos B, H`

Cria o perfil de um jogo: variáveis, faixas, ações e proibições. Importa de fontes públicas, valida contra o jogo e publica versões.

Plataforma (rotina)

`sistema · ator principal nos módulos A, D, F, G`

O que o produto faz sozinho: ler, validar, escrever, registrar, medir, detectar falha.

Serviço de linguagem

`sistema externo · participa de C, G`

O modelo que decide ação e motivo a partir do estado resumido. Trocável por configuração; nunca guarda estado, nunca toca memória.

Emulador

`sistema externo · participa de A, G`

O PCSX2 rodando o jogo do jogador. Expõe a memória pela interface oficial; o produto não o modifica.

Fontes de mapas

`sistema externo · participa de B, H`

Bases públicas de endereços de memória por jogo, mantidas por comunidades. Cobertura e termos de uso variam por título.

DOIS OBJETOS QUE NÃO SE MISTURAM

Perfil de jogo e sessão viva

Perfil de jogo

Imutável durante a sessão, versionado. É o que o jogo é para o produto: identificação do título, variáveis com significado e faixa, catálogo de ações com limites, lista do que nunca se toca. Um perfil serve a qualquer sessão daquele jogo, em qualquer máquina.

Sessão viva

Mutável. É o que de fato aconteceu numa partida: cada observação, cada decisão do mestre inclusive "nada", cada escrita com estado antes e depois, cada pausa e reversão do jogador. É a fonte de verdade para reverter, retomar e verificar.

O mestre pode surpreender, como um mestre humano faz. A diferença é que toda surpresa fica registrada com motivo e dentro do perfil: isso é o que separa dirigir de trapacear, e é o que permite testar o mestre em vez de só opinar sobre ele.

REGRAS DE NEGÓCIO

Dez regras que valem em todos os casos de uso

As cinco primeiras são as regras de mesa: o que mantém a diversão quando o mestre intervém. As demais são regras do produto, e quatro delas nascem de falhas que o protótipo teve de verdade. Cada uma vira teste.

RN-01 **Toda intervenção tem motivo e registro**

Estado antes, ação, estado depois e o motivo do mestre ficam gravados na sessão viva. Intervenção sem registro não existe. Obtemos: mestre auditável.

módulos C, D, F

RN-02 **Dificulta, não derrota**

O mestre pode endurecer, tirar recurso e criar aperto. Não pode zerar a vida nem causar derrota direta. Obtemos: desafio sem roubo.

módulos C, D

RN-03 **Progresso e save são intocáveis**

História, nível, itens-chave, arquivo de save e cartão de memória nunca entram no catálogo de ações. Recursos consumíveis entram, dentro da faixa. Obtemos: conquista com valor.

módulos B, D

RN-04 **O jogador vê e pode desfazer**

Ação e motivo aparecem no overlay no momento em que acontecem. A última ação pode ser revertida e a sessão pausada a qualquer instante. Obtemos: o controle fica com o jogador.

módulos D, E

RN-05 **Não agir é ação válida**

Há intervalo mínimo entre intervenções e "nada" é uma decisão registrada como qualquer outra. Obtemos: ritmo, não ruído.

módulos C, F

RN-06 **O mestre só age pelo perfil**

O catálogo de ações é derivado do perfil, nunca do texto do serviço de linguagem, e todo valor escrito fica dentro da faixa declarada. O protótipo somava dinheiro sem teto. Obtemos: nenhuma escrita cega.

módulos B, C, D

RN-07 **Serviço de linguagem nunca toca memória**

Ele devolve ação e motivo; só a plataforma lê e escreve, validando contra o perfil. Obtemos: separação testável sem modelo.

módulos A, C, D

RN-08 **Decidir sobre o estado atual**

A plataforma relê a variável antes de escrever. Se mudou fora do esperado desde a decisão, a ação é descartada e registrada como tal. O protótipo agia sobre um estado lido segundos antes. Obtemos: nenhuma ação sobre estado velho.

módulos C, D, G

RN-09 **O jogo é do jogador**

Imagem do jogo e BIOS ficam na máquina dele; nunca entram no repositório, nos testes compartilhados nem em qualquer envio. Ao serviço de linguagem só vai o estado resumido pelo perfil. Obtemos: respeito à propriedade e à licença.

módulos A, G

RN-10 **Falha é visível e silencia o mestre**

Serviço fora do ar ou resposta inválida aparece no overlay; sem decisão válida, nenhuma escrita acontece; o jogo segue sem o mestre e a sessão retoma. O protótipo ficou meses parecendo vivo com o serviço desligado. Obtemos: nunca parece vivo estando morto.

módulos E, G

ORIGEM

O que o protótipo provou e o que ensinou

Entre fevereiro e março de 2026 o cliente construiu um protótipo sobre Final Fantasy XII no PCSX2: um overlay que lia vida, energia, dinheiro e atributos do personagem direto da memória do emulador e, a cada ciclo, pedia a um serviço de linguagem uma ação de mestre. Vinte scripts, a maioria ferramentas de varredura de memória.

PROVOU

- Ler e escrever a memória do jogo de fora do emulador funciona, em tempo real, sem modificar o emulador.
- O ciclo observar, decidir, agir com um serviço de linguagem no meio funciona e produz decisões com motivo.
- Um overlay sobre a janela do jogo é suficiente como interface: estado, última ação, motivo, histórico.
- Deslocamentos relativos entre variáveis do mesmo jogo são estáveis entre sessões: achado um ponto de referência, o resto segue.

FALHOU

- A base da memória era procurada por varredura do valor do dinheiro, com filtros e um teste de escrita que perguntava ao jogador se o valor mudou. Doze scripts para um problema que a interface oficial do emulador resolve.
- Posições de entidades foram caçadas por diferença de movimento: trinta candidatas, duas plausíveis, nenhuma confiável.
- O serviço de linguagem usado foi descontinuado e o código engolia o erro: o overlay seguia vivo decidindo "nada".
- Endereços absolutos de uma sessão ficaram gravados no código; a chave do serviço também.

QUATRO LIÇÕES QUE VIRARAM REGRA

Falha silenciosa
A exceção do serviço virava a decisão "nada" com o erro escondido no motivo. Agora é RN-10: falha visível, mestre em silêncio, nenhuma escrita.

Estado velho
O estado era lido, o serviço demorava, e a escrita usava o valor antigo. Agora é RN-08: reler antes de escrever, descartar se mudou.

Sem teto
Dar dinheiro somava sem limite; atributos tinham tetos fixos no código. Agora é RN-06: faixa no perfil, catálogo derivado do perfil.

Ação dentro do serviço
O serviço escolhia entre ações cujas escritas estavam coladas no mesmo código. Agora é RN-07: o serviço devolve intenção, a plataforma executa e valida.

O que se aproveita. A tabela de deslocamentos do personagem relativa ao endereço do dinheiro, o desenho do overlay e o ciclo de decisão. O que se descarta: toda a varredura de memória, os endereços absolutos, o mapeamento de entidades e a chave gravada no código.

ROTEIRO DE IMPLEMENTAÇÃO

Três documentos, um portão entre cada, depois código

O projeto é conduzido como processo de engenharia, do protótipo ao produto. Nenhuma etapa começa sem a anterior aprovada pelo cliente. O que for aprendido numa etapa volta para o documento anterior como revisão numerada, não como conversa perdida.

1. Mapa do projeto
Este documento: visão, atores, módulos, regras, decisões e lacunas.

→ **Portão**
Aprovação do cliente.

→ **2. Casos de uso**
Uma ficha por caso: objetivo, fluxo, alternativas, exceções, resultado, regras. Entregue módulo a módulo.

→ **Portão**
Aprovação do cliente.

3. Blueprint técnico
Arquitetura, formato do perfil, protocolo entre mestre e plataforma, cards de implementação com critério de aceite.

→ **Portão**
Aprovação do cliente.

→ **4. Código, card a card**
Branch por card, revisão, integração contínua verde, critério de aceite provado.

→ **Portão por fase**
Sessões jogadas pelo cliente.

COMO CADA CARD É FECHADO

- Critério de aceite escrito antes do código, no blueprint.
- Plataforma e perfil testados isolados, sem serviço de linguagem, com memória simulada.
- Mestre testado numa bancada de estados sintéticos em que toda decisão tem de cumprir as regras de negócio.
- Integração contínua verde é o registro de entrega.
- Mudança de requisito entra como caso de uso novo, com código próprio.

COMO A QUALIDADE É MEDIDA

- Registro de defeitos: cada travada ou intervenção estranha em sessão jogada vira item com causa.
- Medir antes de corrigir: decisão, custo e latência têm número antes de qualquer ajuste.
- Revisão pós-fase: o que funcionou, o que falhou, o que muda no processo.
- Lista de lacunas fechada e republicada no fim de cada fase.

O ESQUELETO AMBULANTE

O primeiro código entregue é a fatia mais fina de ponta a ponta: a ponte pela interface oficial do emulador, o perfil mínimo de Final Fantasy XII com vida e dinheiro, o mestre com três ações e "nada", o overlay e o registro. Ele prova a arquitetura antes de qualquer funcionalidade ganhar corpo, e é o primeiro item que o cliente joga. O que o destrava é um único dado: o endereço fixo do dinheiro na versão do jogo que o cliente usa (lacuna LC-01).

FICA PARA O BLUEPRINT TÉCNICO

- Linguagem e estrutura do repositório; o protótipo é em Python e isso é um ponto de partida, não uma decisão.
- Qual das duas pontes com o emulador, decidida por medição de latência e robustez, nunca por preferência.
- Qual serviço de linguagem e qual modelo, decididos com medição de custo, latência e qualidade de decisão, nunca por preferência.
- Formato exato do perfil de jogo e da sessão viva, e como o mapeador valida um perfil.
- Protocolo entre mestre e plataforma, o ciclo de decisão e o que acontece quando o mestre pede algo fora do catálogo.

FASES DE ENTREGA

O que entra em cada fase

A fase 1 é o mestre jogável com um perfil. A fase 2 abre o produto para outros jogadores e outros jogos, e traz as entidades. A fase 3 dá olhos e mãos ao mestre, só sobre o que as duas anteriores provaram.

Fase 1 · mestre jogável com um perfil

módulos A a G

Abrir o jogo no emulador, abrir a sessão com o perfil de Final Fantasy XII, jogar com o mestre intervindo em valores numéricos dentro das regras de negócio, ver, pausar, reverter, encerrar. Termina quando o cliente joga uma sessão inteira com toda intervenção registrada e nenhuma fora do perfil.

- A Ponte pela interface oficial
- A Identificar o jogo em execução
- B Perfil à mão, validado contra o jogo
- B Versão do perfil
- C Observar e decidir
- C Tom e intensidade
- D Catálogo e faixa
- D Reverter e pausar
- E Overlay
- E Resumo da sessão
- F Registro reproduzível
- G Falha visível
- G Custo e latência

Fase 2 · outros jogadores, outros jogos, entidades

módulos H, I

Perfis compartilháveis, importação de fontes públicas de mapas, descoberta assistida de variáveis com o serviço de linguagem como ajudante do mapeador, segundo e terceiro jogo com perfil, e o primeiro jogo com mapa de entidades. Só começa com a fase 1 jogada, revisada e com a validação da lacuna LC-08 feita.

- H Publicar e importar perfil
- H Importar de fontes públicas
- H Descoberta assistida
- I Posição de entidades
- I Aproximar, afastar, reunir
- – Instalação para outros jogadores

Fase 3 · visão e controle

módulo J

O mestre passa a ver a tela do jogo e a acionar o controle por um gamepad virtual. É outro tipo de intervenção, com outras regras, e só entra sobre perfis e registro já provados. As regras de negócio continuam valendo: tudo visível, tudo registrado, nada de derrota direta.

- J Captura da tela para o mestre
- J Controle virtual
- J Regras próprias de intervenção por controle

Jogo de abertura. Final Fantasy XII, na versão que o cliente já usa, com o perfil montado a partir dos deslocamentos que o protótipo levantou. Para testes automáticos e integração contínua, uma memória simulada com o mesmo formato do perfil, sem jogo nenhum, para que a plataforma e o mestre rodem sem emulador e sem serviço de linguagem.

DECISÕES

Três fechadas, nove propostas

As três primeiras foram respondidas pelo cliente em 26 de outubro de 2026. As demais são propostas deste documento e fecham com a aprovação dele. Mudança posterior entra como revisão numerada, com data, e propaga para os casos de uso e para o blueprint.

CÓDIGO	DECISÃO	ESCOLHA	POR QUÊ	ESTADO
DC-01	Usuário final	Outros jogadores também, não só o cliente	Define desde já perfis compartilháveis e a separação entre quem mapeia e quem joga.	fechada 26/10
DC-02	Escopo da fase 1	Só valores numéricos; entidades na fase 2; tela e controle na fase 3	Valores numéricos têm gabarito e fontes públicas; entidades exigem engenharia reversa por jogo.	fechada 26/10
DC-03	Visibilidade	Transparente: overlay com ação e motivo, pausa e reversão	O jogador mantém o controle; é o que torna o mestre aceitável em vez de intrusivo.	fechada 26/10
DC-04	Papel da inteligência artificial	Mestre invisível que molda a partida	Não é jogador por inteligência artificial nem trainer com conversa; é o produto do protótipo.	proposta
DC-05	Alvo	Qualquer jogo de PlayStation 2 no PCSX2; Final Fantasy XII é o banco de ensaio e o perfil número 1	O genérico está na ponte e no formato do perfil; o específico fica dentro de cada perfil.	proposta
DC-06	Ponte com o emulador	Interface oficial do emulador ou símbolo exportado da memória; varredura por valor nunca mais	Elimina os doze scripts do protótipo e torna os endereços do jogo constantes.	proposta
DC-07	Emulador	Só PCSX2 na fase 1; outros emuladores e consoles fora até o formato do perfil estar provado	Um emulador, uma ponte, uma fonte de variação a menos.	proposta
DC-08	O jogo é do jogador	O produto não distribui jogo nem BIOS; imagem de jogo nunca entra no repositório	Propriedade e licença são do jogador; o produto só lê memória em execução.	proposta
DC-09	O jogador não pede ação	Configura tom e intensidade antes da sessão; durante, só pausa, reverte e encerra	Se o jogador escolhe o efeito, é trainer. Transparência com pausa e reversão é o limite que mantém o mestre.	proposta
DC-10	Idioma	Produto e perfis em português do Brasil; rótulos podem manter o nome original do jogo	Público do cliente; o nome original evita ambiguidade no mapeamento.	proposta
DC-11	Nome, pasta e repositório	"IAEmPS2" (provisório); pasta Projetos-SSD\iaemps2; repositório privado, branch e merge, nunca commit direto na principal	Mesmo esquema de trabalho dos outros projetos do cliente.	proposta
DC-12	Origem do perfil	À mão, importado de fonte pública ou descoberta assistida; qualquer origem passa por validação contra o jogo em execução antes de entrar numa sessão	Perfil errado escreve no lugar errado; a validação é a mesma independente da origem.	proposta

LACUNAS

O que ainda não está resolvido

Lista republicada no fim de cada fase. Item fechado sai com a data e o documento em que foi resolvido. As três primeiras precisam de um dado do cliente ou de uma verificação na máquina dele.

CÓDIGO	LACUNA	ONDE RESOLVE	RISCO SE FICAR ABERTA
LC-01	Versão e serial de Final Fantasy XII em uso. Os deslocamentos do protótipo são relativos ao endereço do dinheiro; falta o endereço fixo desse valor na versão do cliente, que é um dado só.	Antes do esqueleto ambulante, com o emulador aberto	Esqueleto não anda.
LC-02	Confirmar que o PCSX2 instalado expõe a memória pela interface oficial ou pelo símbolo exportado. Confirmado só no código-fonte do emulador, não no executável do cliente.	Primeira tarefa do módulo A	Ponte não funciona; volta a varredura.
LC-03	Como identificar o jogo em execução (serial, soma de verificação) pela ponte, para casar com o perfil certo.	Casos de uso do módulo A	Perfil de um jogo escrevendo na memória de outro.
LC-04	Cobertura real e termos de uso das fontes públicas de mapas, jogo a jogo.	Casos de uso do módulo B; levantamento na fase 2	"Qualquer jogo" vira "poucos jogos".
LC-05	Como medir a qualidade do mestre sem gabarito. Proposta: bancada de estados sintéticos em que toda decisão tem de cumprir as regras de negócio, mais sessões jogadas; a diversão em si continua julgamento do jogador.	Blueprint, seção de testes do mestre	Sem medida, regressão passa despercebida.
LC-06	Custo e latência por sessão no ciclo de decisão, e limite aceitável para o cliente.	Medição no esqueleto ambulante	Produto inviável de operar.
LC-07	Levantamento do que já existe: produtos que alteram jogos em tempo real por evento externo, em emulador ou não. Nenhuma pesquisa feita ainda.	Antes da fase 2	Construir o que já existe.
LC-08	Validação com outros jogadores antes do catálogo de perfis: medir interesse e qual jogo pedem primeiro.	Fim da fase 1, antes de qualquer trabalho da fase 2	Produto certo para uma pessoa, errado para o resto.
LC-09	Nome definitivo do produto.	Decisão do cliente, a qualquer momento	Nenhum.

Como usar este documento no dia a dia. Ele é a origem de tudo: os casos de uso detalham os módulos daqui, o blueprint implementa os casos de uso, e cada card de código aponta para o caso de uso que cumpre. Quando uma necessidade nova surgir, ela entra primeiro aqui, como módulo, regra, decisão ou lacuna com código próprio, e só depois desce. Assim o escopo continua visível e negociável, e nenhuma decisão fica só na conversa.